

Module 8 — Integration Task: RAG Service

In this integration task you will assemble a complete RAG mini-service that exposes

`rag_pipeline(query) -> dict`: retrieve top-k context from Weaviate using the lab's hybrid retriever, build a context-injected prompt, generate an answer with `flan-t5-base`, and evaluate the end-to-end pipeline on a 30-question evaluation set with two metrics — `answer-keyword recall` on `answerable` questions and `abstention rate` on `borderline` ones.

This integrates the lab's retrieval layer with a generator and adds the second metric paradigm (`groundedness`) on top of the lab's retrieval metrics.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Read this first — what you're being graded on

This integration is graded on **pipeline construction**, **methodology adherence**, **metric-split correctness**, and **interpretation** — not on whether you achieve a high `answer_keyword_recall_main` number.

Why this matters up front: under the canonical methodology specified below — `flan-t5-base` with greedy decoding, the exact abstention prompt, and answer-only context — `answer_keyword_recall_main` will land in the range **-0.00 to -0.10** on the 25 answerable rows. **This is expected** as a measured property of `flan-t5-base` under the abstention prompt: the model obtains on 21–23 of 25 answerable rows even when the retrieved context contains the answer. It is not a sign that you implemented the pipeline incorrectly.

We probed `flan-t5-large` under the same methodology to verify this is real and not a small-model issue: the larger model regressed on this metric to 0.00 (attempting only 3/25 rows v.s. base's 4/25). The over-abstention is structural to this model family under this prompt.

If your canonical run lands anywhere in the -0.00-0.10 range, your pipeline is correct. The Tier 1 challenge formalizes the investigation of what would lift this number — prompt iteration, context iteration, generator iteration — but is opt-in. The base assignment is graded on the four bullet points above.

The TA rubric reflects this: **methodology adherence is weighted 20%** (more than `groundedness`, more than the `retrieve` or `test` methods) because the bug class this module introduces — wrong prompt, wrong alpha, wrong model — produces plausible-looking output that silently degrades the metric. Stay faithful to the canonical config.

What You Will Produce

Committed to your repo:

- `rag_service.py` — your implementation of `retrieve`, `build_prompt`, `generate`, `rag_pipeline`, `groundedness_score`, `evaluate`, `rag`
- `tests/test_rag.py` — at least three tests you wrote yourself (the starter ships `pytest.fail(...)` stubs; you replace them with real assertions)
- `README.md` — pipeline overview, how to run it, your eval results, known limitations.

The starter repo bundles the same Stack Exchange Q&A corpus as the lab (`data/corpus.jsonl`, ~1,000–1,500 posts), the integration's evaluation set (`data/rag_eval.jsonl`, 30 questions), and helper code (`index_helpers.py`) so you don't have to re-implement ingestion.

Setup

Submission flow — all Module 8 assignments. Module 8 uses a fork-and-submit flow (drill, lab, integration, both stretches) instead of GitHub Classroom. Steps 1–3 below are the new part; steps 4–8 are the same workflow you used in the drill and lab.

- Fork the template repo to your GitHub account.** Open github.com/LevelUp-Applied-AI/m8-8 and click **Fork** (top right) — set the owner to your personal GitHub account — click **Create fork**. Leave "Copy the main branch only" checked.
- Enable Actions on your fork.** On your forked repo, click the **Actions** tab. GitHub disables workflows on forks by default — click **I understand my workflows, go ahead and enable them**. (One-line per fork. The autograder cannot run on your PR until this is done.)
- Clone your fork (not the upstream repo):**

```
git clone https://github.com/your-github-username/m8-18.git
cd m8-18
```

- Install dependencies (cached from the lab — should be fast):**

```
pip install -r requirements.txt
```

- Make sure Weaviate is running:**

```
docker ps                                # should list the 'weaviate' c
curl http://localhost:8080/v1/.well-known/ready # should return 200
```

- Re-ingest the corpus (the integration repo is isolated from the lab — Weaviate state is not assumed):**

```
python -c "from index_helpers import index_corpus_if_needed; from sentence_to"
```

This is idempotent — if the `Post` class already exists with the right object count, the helper is a no-op.

- Create your working branch:**

```
git checkout -b integration-rag-service
```

All implementation work goes in `rag_service.py`. The starter file contains function signatures, docstrings, the `STOPWORDS` constant, and the `ABSTAIN_PHRASES` constant.

Methodology Specifications

These are the exact contracts the autograder enforces and the TA rubric grades against. Stay faithful to them.

- Top-k:** `k = 5`. Hybrid retrieval with `alpha = 0.5` (delegate to the lab's `hybrid_search`).
- Prompt template (exact, byte-for-byte):**

```
Answer the question using only the context. If the context does not contain t

Context:
[1] [title_1]: [answer_text_1]
[2] [title_2]: [answer_text_2]
[3] [title_3]: [answer_text_3]
[4] [title_4]: [answer_text_4]
[5] [title_5]: [answer_text_5]

Question: {query}
Answer:
```

Each `[answer_text_i]` is the retrieved post's `answer_text` field, truncated to its first **80 whitespace-split tokens**. Title and `[i]:` markers are kept to give `flan-t5-base` orienting cues and create a citation-like structure.

- Why `answer_text` and not `text` (the full doc)?** The question portion of each retrieved post is largely redundant with the learner's query (both ask about the same topic) and consumes context budget that should go to the answer content. Truncating the full `text` to 80 tokens often leaves the prompt with five copies of the question and never reaches the answer — a structural under-extraction bug. Always slice from `answer_text`.
- Generator:** `google/flan-t5-base`, `tokenizer = AutoTokenizer.from_pretrained(...)`, `model = AutoModelForSeq2SeqLM.from_pretrained(...)`. **Load both at module level — not inside `generate()`**. Per-call loading is a 40× slowdown.
- Generate call:** `model.generate(input_ids=..., attention_mask=..., max_new_tokens=128, num_beams=1)`. Greedy decoding (`num_beams=1`) is the contract for reproducibility on CPU.
- Tokenizer call:** `tokenizer(prompt, return_tensors="pt", truncation=True, max_length=512)`.
- Row-class definitions in `data/rag_eval.jsonl`:** A row is "answerable" if `difficulty ∈ {single_fact, single_doc_synthesis}` — 25 rows. A row is "borderline" if `difficulty ∈ {"borderline"}` — 5 rows. Borderline rows are constructed so the supporting document does not contain the answer; the correct model behavior is to abstain.
- `answer_keyword_recall_main` —** computed over the **25 answerable rows only**. For each row, compute `(number of expected answer keywords appearing in the answer, case-insensitive whole-word match) / (total expected answer keywords)`. **Average across the 25 rows. Borderline rows are excluded** from this metric.
- `borderline_abstain_rate` —** computed over the **5 borderline rows only**. A row is "correctly abstained" if either
 - (a) The lowercased answer contains any phrase from `ABSTAIN_PHRASES = ["I don't know", "I do not know", "not in the context", "the context does not", "cannot be answered", "no information"]`, or
 - (b) `len(answer.strip()) ≤ 20 characters AND groundedness_score(answer, contexts) ≤ 0.2`.

The two-part disjunction is intentional. Phrase-only abstention misses some valid refusals (e.g., "The retrieved passages don't say"), `groundedness-only` abstention can be fooled by a short hallucinated answer that accidentally shares stopword-stripped tokens with the context. Both clauses are needed to avoid those blind spots. Higher rate = better. Reported in [0, 1].

- `mean_groundness_main` —** `groundedness_score` averaged over the 25 answerable rows.
- `mean_groundness_borderline` —** `groundedness_score` averaged over the 5 borderline rows. Reported separately from `_main` — never averaged in.
- `groundedness_score` —** see Task 6 below.
- `per_question` —** list of one dict per eval row containing at minimum `["row_index", "difficulty", "question", "answer", "groundedness", "matched_keywords"]` for answerable rows and `["row_index", "difficulty", "question", "answer", "groundedness", "abstained"]` for borderline rows. Reserved for diagnostic inspection; not aggregated.

Task 1: Stand Up Weaviate and Re-Ingest the Corpus

The integration repo is isolated. Use the provided `index_helpers.index_corpus_if_needed(client, corpus_path, embedder)` to populate Weaviate. The helper is idempotent — if the `Post` class already exists with the right object count, it does nothing. Run the one-liner from Setup step 5.

You do not re-implement ingestion in this assignment. Your work starts at retrieval.

Task 2: Implement `retrieve(query, k=5)`

```
def retrieve(query: str, k: int = 5) -> list[dict]:
    """
    Wrap the lab's hybrid_search with alpha=0.5. Return enriched dicts ~
    not just doc_ids ~ because the generator needs the answer content.

    Returns:
        list[dict]: each dict has keys "doc_id", "title", "answer_text".

    """
    # TODO: hybrid_search returns 10x here you also need title and answer_text
    # TODO: query Weaviate for those properties on the same retrieval call
    # TODO: return list[{"doc_id", "title", "answer_text"}]
```

Note: you can either reuse `hybrid_search` and do a second lookup for the properties, or call `client.query.get(query, ["doc_id", "title", "answer_text"]).with_hybrid(..., do)` directly in one call. The second approach is cleaner.

Task 3: Implement `build_prompt(query, contexts)`

Produce the exact template from the methodology spec. Truncate each context's `answer_text` to its first 80 whitespace-split tokens **before** substituting into the template.

```
def build_prompt(query: str, contexts: list[dict]) -> str:
    """
    Return the methodology-canonical prompt string.

    Each context's answer_text is truncated to its first 80 whitespace-split
    tokens. The line "[i] [title]: [answer_text]" is repeated for each
    context i in 1..len(contexts).

    """
    # TODO: build the instruction line + Context block + Question + Answer;
```

The autograder asserts:

- The instruction line appears verbatim.
- Each context line follows `[i] [title]: [answer_text]` shape.
- The truncation is applied to `answer_text`, not to the full `text` field. Truncating `text` instead would re-introduce the structural under-extraction bug — explicitly tested.
- The trailing `Answer:` is present.

Task 4: Implement `generate(prompt)`

Load the model and tokenizer once at module level (top of the file, outside any function). The `generate` function reuses them on every call.

```
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

# Module-level = loaded once on import
_TOKENIZER = AutoTokenizer.from_pretrained("google/flan-t5-base")
_MODEL = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-base")

def generate(prompt: str) -> str:
    """Run the prompt through flan-t5-base with greedy decoding."""
    # TODO: tokenize with truncation=True, max_length=512
    # TODO: model.generate(..., max_new_tokens=128, num_beams=1)
    # TODO: decode with skip_special_tokens=True; return string
```

Task 5: Implement `rag_pipeline(query, k=5)`

Compose the four steps:

```
def rag_pipeline(query: str, k: int = 5) -> dict:
    """Full RAG: retrieve -> build_prompt -> generate -> return."""
    # TODO: contexts = retrieve(query, k)
    # TODO: prompt = build_prompt(query, contexts)
    # TODO: answer = generate(prompt)
    # TODO: return {"query", "answer", "contexts", "prompt"}
```

Task 6: Implement `groundedness_score(answer, contexts)`

Per the methodology spec:

- Lowercase both `answer` and the concatenation of `contexts[i]["answer_text"]` (the same text shown to the generator).
- Tokenize on `whitespace + punctuation`.
- Remove stopwords using the `STOPWORDS` constant in the starter (~50 common English words). No external library.
- Compute `(answer_tokens ∩ context_tokens) / (answer_tokens)`.
- Empty-answer edge case: return 0.0 if the answer is empty after stopword removal.

```
def groundedness_score(answer: str, contexts: list[dict]) -> float:
    """Content-word overlap of answer over concatenated context. Returns [0, 1]."""
    # TODO: tokenize answer and concatenated answer texts
    # TODO: drop STOPWORDS; lowercase
    # TODO: (intersection) / (answer_tokens), with 0.0 fallback for empty answer
```

Task 7: Implement `evaluate_rag(eval_path)`

Iterate the eval pairs in `data/rag_eval.jsonl`. Partition rows by `difficulty`. Compute the four headline metrics per the methodology spec, plus `per_question`.

```
def evaluate_rag(eval_path: str) -> dict:
    """
    Run the pipeline over the 30 eval rows and return the five-key dict:

    {
        "answer_keyword_recall_main": float, # over 25 answerable rows
        "borderline_abstain_rate": float, # over 5 borderline rows
        "mean_groundness_main": float, # over 25 answerable rows
        "mean_groundness_borderline": float, # over 5 borderline rows
        "per_question": [ {... }, ... ], # 30 rows
    }

    """
    # TODO: load eval pairs
    # TODO: for each row; result = rag_pipeline(question)
    # TODO: tag row by difficulty; compute per-row metrics
    # TODO: aggregate by row class; return dict
```

The autograder asserts all five keys are present and that `per_question` has length 30 with `difficulty` tagged on each row. Folding the borderline rows into a single `answer_keyword_recall` over all 30 rows would penalize correct abstention — the autograder explicitly checks for the split.

Task 8: Write Your Tests

The starter ships `tests/test_rag.py` with three test functions whose bodies are `pytest.fail("Not implemented – write your test here")`. Replace each with a real assertion. The required test functions:

- `test_retrieve_returns_k_or_fewer` — `retrieve("any query", k=5)` returns a list of dicts of length ≤ 5; each dict has `doc_id`, `title`, `answer_text` keys.
- `test_build_prompt_contexts_correct` — given a fixture query and 5 mocked context dicts, the prompt string contains the instruction line, the `[1] ... [5]` numbered context block, and the trailing `Answer:`.
- `test_groundness_zero_for_unrelated_answer` — `groundedness_score("xyzzy plugh quux", [{"answer_text": "the quick brown fox"}]) = 0.0` (no overlap, ≤ 0.1).

The autograder runs an AST-based meta-test (`test_learner_tests_complete`) that verifies each test function has at least one `assert` node and no remaining `pass` or `pytest.fail("Not implemented")` stub. Writing `assert True` would technically pass the AST check — but the test then has no signal value, which the TA rubric will catch.

Task 9: Write the README

`README.md` should cover, in order:

- Overview** — one paragraph on what the service does.
- Setup** — how to bring up Weaviate, install dependencies, ingest the corpus.
- How to run it** — example `rag_pipeline(...)` call and what the response shape looks like.
- Eval output** — paste your `evaluate_rag` result. Include the four headline metrics and one or two example rows from `per_question`.
- Known limitations** — at least three. The first one is freebie: write a sentence about the `answer_keyword_recall_main` baseline (see the framing block at the top of this file). Other limitations to consider: `flan-t5-base`'s 512-token input ceiling; `groundedness_score`'s blindness to paraphrase; the `STOPWORDS` list's English-only scope.

The autograder asserts the README is at least 400 characters and mentions the words "RAG" and "groundedness". The TA rubric grades it on substantive coverage of those five sections (10% of the integration grade).

Submitting

When all functions are implemented, autograder is green, your tests have real assertions, and the README is written:

- Run the autograder locally:**

```
pytest tests/ -v
```

All eleven autograder tests should pass.

- Commit and push:**

```
git add rag_service.py tests/test_rag.py README.md
git commit -m "Integration 8: RAG service with answer-aware methodology"
git push origin integration-rag-service
```

- Open a Pull Request within your fork** — base branch `main`, compare branch `integration-rag-service`. **Important:** the PR must stay inside your fork. When you click "Compare & pull request", GitHub defaults to "base repository: LevelUp-Applied-AI/m8-18" — change the base repository dropdown to your own fork (<your-github-username>/m8-18) before opening the PR. The autograder runs against your branch; check the CI status on the PR page to confirm green. If you see "Workflows are disabled" on the Actions tab, you missed Setup step 2 — enable them and push an empty commit (`git commit --allow-empty -m "ci: trigger"`) to re-run. Your PR description must include
 - The full `evaluate_rag` output: `answer_keyword_recall_main`, `borderline_abstain_rate`, `mean_groundness_main`, `mean_groundness_borderline`.
 - Two example questions with their generated answers and `groundedness` scores — pick one with a high `groundedness` score and one with a low score to show the spread.
 - A 1–2-sentence interpretation of the four headline metrics: what does this combination tell you about retrieval, generation behavior, `groundedness` and abstention discipline?
 - Paste your PR URL into Talent.LMS → Module 8 → Integration 8 to submit this assignment.

Challenge Extensions

Optional. The base integration is complete without them. The Tier 1 challenge is the formal home for 'but how do I fix `answer_keyword_recall_main`?' experimentation — keep it strictly separate from the base submission.

Tier 1 — Confidence-Aware Answers + Baseline Investigation

The canonical baseline `answer_keyword_recall_main` is -0.00-0.10. This is not a bug — it reflects how `flan-t5-base` behaves under the abstention prompt with answer-aware context. This tier asks you to (a) measure the over-abstention quantitatively and (b) explore at least one intervention.

Submission structure — keep canonical and intervention work strictly separate. The Tier 1 submission consists of two distinct artifacts:

- Canonical baseline implementation** (`stretch-tier-1-report.md`) that, when invoked with the canonical config (`flan-t5-base`, hybrid `alpha=1`, `answer_text` (80) context, exact prompt template, `flan-t5-base`, greedy). TAs grade this against the canonical methodology; do not count intervention experiments toward base methodology adherence. Tier 1 work is additive and must not break or replace the base.
- Tier 1 experimental report** (`stretch-tier-1-report.md` or similar — separate file). Contains the unsure-flag implementation results, the borderline precision/recall, and the intervention experiment.

Steps:

- Modify the pipeline to return an `"unsure"` flag when either (a) `groundedness_score < 0.4`, or (b) the generator emits a phrase from `ABSTAIN_PHRASES`. Implement this as an **optional pipeline variant** (a separate function or a CLI flag) — do not change the canonical `rag_pipeline` signature.
- Compute the `unsure_rate` over the 25 answerable rows. Expected canonical baseline: ~0.85–0.92.
- Compute the **precision/recall of the unsure flag against the labeled difficulty** — "borderline" rows as the gold. Recall = how often the model correctly flags borderline rows; precision = how often unsure-flagged rows are actually borderline. A pipeline that abstains universally has perfect recall but poor precision.
- Investigation deliverable — 150–250 words in `stretch-tier-1-report.md`: experiment with one of these interventions and report the lift on `answer_keyword_recall_main`.
 - Prompt iteration** — soften or rewrite the abstention clause; add a 1+2-example few-shot prefix.
 - Context iteration** — vary top-k between 3/5/8; vary the `answer_text` truncation budget between 60/80/120 whitespace tokens; pull from `text` instead of `answer_text`.
 - Generator iteration** — try a different CPU-runnable model (`flan-t5-small` or `flan-t5-large`); report the trade-off with `borderline_abstain_rate`.
- Report: chosen intervention, measured before/after on the four key dict, qualitative notes on what the trade-off looks like. **Honest reporting of negative results is acceptable and expected** — the project owner's own probe of `flan-t5-large` produced a regression under identical methodology, which is itself useful learning.

Tier 2 — Citation-Style Output

- Modify the prompt to require numbered citations: the generator emits, e.g., "Use 'git rebase -i' to squash commits [2]".
- Parse citations from the generator output and verify each `[i]` is a valid context index (i.e., `1 ≤ i ≤ len(contexts)`).
- Add a `citation_validity` metric: `(valid citations) / (total citations emitted)`.

Tier 3 — Two-Hop RAG

- Implement a two-hop pipeline: the first retrieval surfaces an entity (extracted from the top result); the second retrieval queries on that entity's name to find related context.
- Manually pick 5 multi-hop questions from the corpus where single-hop retrieval fails. Show single-hop v.s. two-hop on those 5.
- Discuss in a short writeup: when does the second hop help, and when does it amplify retrieval errors?

